

JavaWeb期末总结

选择填空

水平线标签

```
<hr>
```

重点：单标签

属性：

- size :水平线的宽度，单位为像素。
- width:水平线的长，默认为页面长度，单位默认为像素，也可使用百分制。
- align：水平线的对齐方式，常用left,center,right.
- noshade：线段无阴影属性，没有属性值。
- color：线段内部属性。

字体设计标签

```
<font> </font>
```

属性：

- size：用来设置字体大小。写法一：“size=X”,X越大字体越大，X取值为1-7，3为默认大小。 写法二：“size=+X”或者“-X”。
- face:用来设置字体类型，默认为宋体。
- color:用来设置字体颜色。

常见文字风格：

```
<b>内容</b>    //设置为粗体
<u>内容</u>    //设置下划线
<i>内容</i>    //设置为斜体
<sup>内容</sup> //设置为上标签
<sub>内容</sub> //设置为下标签
<blink>内容</blink> //设置为闪烁
```

列表标签

```
<ul>内容</ul> //无序列表标签
<ol>内容</ol> //有序列表标签
<li>列表项</li>
```

表格标签

```
<table>内容</table>    //定义表格
<caption>内容</caption>  //定义标题
<tr></tr>    //定义表行
<th></th>    //定义表头
<td></td>    //定义表元
```

Json

什么是JSON？

- JSON 指的是 JavaScript 对象表示法 (JavaScript Object Notation)
- JSON 是轻量级的文本数据交换格式
- JSON 独立于语言：JSON 使用 Javascript语法来描述数据对象，但是 JSON 仍然独立于语言和平台。JSON 解析器和 JSON 库支持许多不同的编程语言。目前非常多的动态（PHP，JSP，.NET）编程语言都支持JSON。
- JSON 具有自我描述性，更易理解

JSON 语法规则

- 数据为 键/值 对。
- 数据由逗号分隔。
- 大括号保存对象
- 方括号保存数组

JSON与JavaScript转换

- JSON.parse()用于将一个 JSON 字符串转换为 JavaScript 对象。
- JSON.stringify()用于将 JavaScript 值转换为 JSON 字符串。

JavaScript Array（数组） 对象

数组对象的作用是：使用单独的变量名来存储一系列的值。

- 删除数组的最后一个元素 - pop()
- 数组的末尾添加新的元素 - push()
- 删除数组的第一个元素 - shift()
- 从一个数组中选择元素 - slice()
- 合并两个数组 - concat()
- 数组排序- sort()

jQuery

#id 选择器

```
$("#id")    // #id 选择器选取带有指定 id 的元素。
```

.class 选择器

```
$(".class")    // .class 选择器选取带有指定 class 的所有元素。
```

:first

```
$("#p:first") //第一个 <p> 元素
```

:first-child

```
$("#p:first-child") //属于其父元素的第一个子元素的且是 <p>的所有元素
```

###

JSP程序段

```
<% =i %> //表达式脚本，将=后面的表达式计算并输出
<% out.println(str); %> //换行加分号；，不换行不加
<% String str=request.getParameter("number") %> //request对象获取请求信息的方法
request.getParameter()
JSP代码段正确用法
<% Java代码段 %>
```

JSP指令

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
//只有contentType后面需要加分号； 该程序段要求可以完整默写

<% @ page import="java.util.Date" language="java"
contentType="text/html";charset="gb2312"%>
```

JSP内置对象session及其它对象

```
//顺序
application > session > request > page > pageContext

//文件的引入格式
<%@ page language="java" import="java.util.*" pageEncoding="gb2312"%> //

//session对象的使用
//向session中放入一个集合对象
ArrayList books = new ArrayList();
//向books里添加要买的书，此处利用add函数即可实现，代码略
session.setAttribute("books",books);//存入books对象
ArrayList books=(ArrayList)session.getAttribute("books") //获取对象

//JSP其他对象
pageContext对象提供了对JSP页面中所有对象及命名空间的访问。
Page对象指向当前JSP程序本身
exception对象获取异常信息，属于java.lang.Exception
config对象在一个JSP程序初始化时JSP引擎向它传递消息
```

Servlet

Servlet是一种运行在服务器端(一般指的是Web服务器)的Java应用程序,且Servlet为多线程,不存在并发问题

Servlet的生命周期

1.init()方法

一个Servlet在服务器上最多只会驻留一个实例,所以说第一次调用Servlet时将会创建一个实例,可以将一些初始化代码放在其中。

2.doGet()/doPost()/service()

客户端对Servlet发送请求,服务器端就会开启一个线程,该线程会调用service()方法,service()方法会根据客户端请求类型来决定调用doGet()还是doPost()。

3.destroy()方法

destroy()在Servlet实例消亡时自动调用。在Servlet消亡之前还必须进行的某些操作,比如释放数据库连接以节省资源等,这时就可以重写destroy()方法了。

Servlet的两种处理请求方法

- doGet()在以get方式请求Servlet运行。常见get方式有链接, get方式表单提交, 直接访问Servlet
- doPost()在以Post方式请求Servlet运行, 常见的post请求为post方式表单提交。

既然JSP和Servlet等价, 在JSP中可以使用内置对象, 那么在Servlet中应该也可以使用内置对象。

- 获得out对象
- 获得request对象和response对象
- 获得session对象
- 获得application对象

常用的Servlet内跳转

- 重定向(对应JSP隐含对象中的sendRedirect())。向前端返回

```
response.sendRedirect("URL地址")
```

- 服务器内跳转(对应JSP隐含对象中的forward())。

```
ServletContext application = this.getServletContext();
RequestDispatcher rd= application.getRequestDispatcher("URL地址")
rd.forward(request,response);
```

JSP和JavaBean

编写JavaBean

```
//在Student.java中完成三个方法
public void setStuno(String stuno){
    this.stuno=stuno
}
public void setStuname(){
    return stuname
}
public void setStuno(String stuname){
    this.stuname=stuname
}
```

在JSP中使用DAO和VO

```
<%@ page language="java" import="java.util.*,java.sql.*" pageEncoding="gb2312"%>
<%@ page import = "dao.StudentDao"%> //调用Dao类访问数据库
<%@ page import = "beans.Student"%>

//先利用DAO类获取到数据库中的Student数组，然后通过JSP的循环获取到每一个Student对象
//获取相关学生信息
<% = student.getStuno() %>
<% = student.getStuname() %>

//使用EL表达式获取学生信息，注意两者区别
${student.stuno}
${sessionScope.student.stuname} //EL表达式通过从session域中获取student对象
```

JSP作用域

applicationScope > sessionScope > requestScope > pageScope

简答题

JSP内置对象

- out对象：负责管理对客户端的输出。
- request对象：负责得到客户端的请求信息。
- response对象：负责向客户端发出响应。
- session对象：负责保存同一客户端一次会话过程中的一些信息
- application对象：表示整个应用的环境信息。（全局）
- exception对象：表示页面上发生的异常，可以通过它获得页面的异常信息。
- page对象：表示的是当前JSP页面本身，就像Java类定义中的this一样。
- pageContext：表示的是此JSP的上下文。
- config对象：表示此JSP的ServletConfig

异步编程

定义

- 1.异步 (Asynchronous, async) 是与同步 (Synchronous, sync) 相对的概念。异步的概念则是不保证同步的概念，也就是说，一个异步过程的执行将不再与原有的序列有顺序关系。
- 2.简单来理解就是：同步按你的代码顺序执行，异步不按照代码顺序执行，异步的执行效率更高。
- 3.异步就是从主线程发射一个子线程来完成任务。

回调函数

回调函数就是一个函数，它是在我们启动一个异步任务的时候就告诉它：等你完成了这个任务之后要干什么。这样一来主线程几乎不用关心异步任务的状态了。

补充

异步回调广泛应用于 AJAX 编程。

MVC

概念：

- M(Model):封装应用程序的数据结构和事务逻辑，集中体现应用程序的状态，当数据状态改变的时候能够在视图里面体现出来。(JavaBean非常适合)
- V(view): 他是Model的外在表现，当模型状态改变时有所体现。(JSP非常适合)
- C(controller):对用户的输入进行响应，将模型和视图联系到一起，负责将数据写到模型中，并调用视图。(java Servlet非常适合)

好处：

- 适合分工：每一个程序员只需要关心自己需要关心的那个模块。
- 维护方便：例如需要修改其中的一个部分，则对相应的模块进行修改就可以了。

JavaScript 计时事件

通过使用 JavaScript，我们有能力做到在一个设定的时间间隔之后来执行代码，而不是在函数被调用后立即执行。我们称之为计时事件。

在 JavaScript 中使用计时事件是很容易的，两个关键方法是：

- setInterval() - 间隔指定的毫秒数不停地执行指定的代码。
- setTimeout() - 在指定的毫秒数后执行指定代码。

代码

表单

1.前端设计提交表单页

```
<div align="center">
    <h1 style="color: red;">课程信息录入</h1>
    <a href="index.jsp">返回主页</a>
    <form action="CourseServlet?method=add" method="post" onsubmit="return
check()">
```

```

<div class="a">
    课程名称<input type="text" id="name" name="name"/>
</div>
<div class="a">
    任课教师<input type="text" id="teacher" name="teacher" />
</div>
<div class="a">
    上课地点<input type="text" id="classroom" name="classroom" />
</div>
<div class="a">
    <button type="submit" class="b">保 存</button>
</div>
</form>
</div>

```

2. 设置表单提交不能为空

```

<script type="text/javascript">
    function check() {
        var name = document.getElementById("name");
        var teacher = document.getElementById("teacher");
        var classroom = document.getElementById("classroom");

        //非空
        if(name.value == '') {
            alert('课程名称为空');
            name.focus();
            return false;
        }
        if(teacher.value == '') {
            alert('教师为空');
            teacher.focus();
            return false;
        }
        if(classroom.value == '') {
            alert('上课地点为空');
            classroom.focus();
            return false;
        }
    }
</script>

```

3. 验证数字是否在某个区间内

```

var patt=/^\d{50,100}$/;
//需要先判断是否为数字

if(!patt.test(classroom.value)){
    alert('输入的数字不在50-100之间')
    return false;
}

```

4.验证输入只能为大小写字母且限制位数

```
var patt1=/^[A-Za-z]+$;/    //判断大小写字母
var patt2=/^.{3}$/;        //限制字符为位数

if(!patt1.test(classroom.value)||!patt2.test(classroom.value)){
    alert('输入的字符不是8位的英文字符')
    return false;
}
```